Autonomous Navigation and Target Reaching with the RosBot in Webots

Khushalkumar Jitendrabhai Variya
Dhruvin Harshadbhai Gopani[1]

[1] *OTH Amberg-Weiden, Study programme “Master of Artificial Intelligence for Industrial Applications”*

July 2026

Abstract

We implemented autonomous navigation for the RosBot in five Webots maze environments. The robot starts without a given map, builds an online occupancy grid, detects blue and yellow target pillars, avoids forbidden green floor regions, and plans safe routes using its own sensor data. The implementation uses wheel encoders, IMU yaw, LiDAR, RGB camera, and depth camera information. The controller is organized into clear modules for sensing, localization, mapping, perception, planning, motion control, recovery, persistence, and visualization. Three controller folders were submitted: one for Maze 1, one for Maze 2, and one shared version for Mazes 3, 4, and 5. In the recorded tests, the robot reached the blue pillar first and then the yellow pillar in all five mazes without using the Webots Supervisor API or changing the provided robot and world files.

Keywords: *autonomous robots, Webots, RosBot, occupancy grid, SLAM, frontier exploration, A* planning, RGB-D perception*

1. Introduction

The goal of this project was to implement autonomous navigation for the RosBot in Webots. After the simulation starts, the robot must first find and reach the blue pillar. Only after reaching the blue pillar is it allowed to navigate to the yellow pillar. The total mission time stops when the yellow pillar is reached.

The main difficulty is that the robot does not receive a complete map at the beginning. While moving, it has to perceive walls, narrow passages, forbidden green ground, floating obstacles, and the two coloured pillars. The controller therefore uses onboard sensors to build its own map and to decide where it can drive.

The solution follows a simple navigation pipeline. The robot reads the sensors, estimates its pose, updates the occupancy map, detects special objects, selects a goal, plans a path, follows the path, and runs recovery if it gets stuck. The same main idea is used in all five mazes, with separate parameter values and small controller changes where the layouts require them.

2. Assignment Requirements and Constraints

2.1. Mission Requirements

The mission is to solve each maze autonomously. The required order is strict: the robot must reach the blue pillar first and then the yellow pillar. A run is complete only when the yellow pillar is reached after blue.

The controller prints the mission time in two parts. The first timer measures simulation start to blue pillar. The

second timer measures blue pillar to yellow pillar. The total time is the full simulation start to yellow pillar time.

2.2. Environment Challenges

The mazes contain normal walls, tight corridors, green floor regions, target pillars, and obstacles that may be difficult to detect with only one sensor. Green floor is forbidden and must be treated as an obstacle. Some obstacles may not be represented reliably by a simple two-dimensional LiDAR endpoint map, so RGB-D camera information is also used.

2.3. Implementation Constraints

The controller follows the assignment constraints. It does not use the Webots Supervisor API, it does not modify the robot model, and it does not modify the provided world files. All navigation decisions are made from the robot sensors and the controller's own internal map. The use of only normal Webots robot-controller access keeps the run conditions consistent with an autonomous robot task [1, 2].

3. Task Decomposition

The project was split into the technical tasks shown in Table 1. The table connects each assignment problem to the implemented controller method.

Table 1. Project tasks and implemented methods.

Project task	Main method	Effect in the controller
Build a map	LiDAR occupancy grid with ray tracing	Walls are marked as occupied, visible free space is cleared, and unknown cells remain available for exploration.
Localize the robot	Wheel odometry, IMU yaw, and scan matching	Encoders estimate movement, the IMU gives heading, and scan matching reduces drift when enough wall evidence is visible.
Avoid green floor	HSV colour mask with camera/depth projection	Green cells are written into a separate map layer and are blocked during planning.
Detect target pillars	RGB colour detection with distance and repeated confirmation	Blue and yellow pillars are accepted as targets only after stable observations.
Handle floating obstacles	RGB-D height filtering and evidence fusion	Obstacles that are weak in the LiDAR map can still be stamped into the planning map after repeated evidence.
Explore the maze	Frontier-based exploration	The robot expands known free space by moving toward boundaries between explored and unexplored areas [3].
Plan a path	A* search on an inflated cost map	The robot searches for a route that avoids walls, green floor, and confirmed obstacles. A* and other mobile-robot planning ideas are standard in robotics references [4, 5].
Move and recover	Path following, safety checks, and staged recovery	The robot follows the route, stops unsafe movement, and tries escape actions if progress becomes too low.

4. Overall Architecture

The controller is organized as a step-by-step navigation system. Sensor readings are converted into pose and map information first. Then the state machine chooses the current mission action, the planner creates a route, and the motion controller sends wheel commands.

4.1. Controller Folders Used for the Five Mazes

The submitted `Maze_Solutions.zip` contains three controller folders. The folder names and the corresponding maze usage are shown in Table 2. The table uses the exact folder names from the submitted archive.

Table 2. Controller folders used for the five mazes.

Maze	Controller folder	Reason for use
Maze 1	`manual_mapping_maze1_solution/manual_mapping/`	Dedicated Maze 1 version. It uses the same mapping, perception, planning, motion, and recovery structure, with values tuned for this layout.
Maze 2	`manual_mapping_maze2_solution/manual_mapping/`	Dedicated Maze 2 version. It contains additional route checking, map cleanup, target handling, and recovery behavior because this run was the longest recorded one.
Mazes 3, 4, and 5	`manual_mapping_maze345_solution/manual_mapping/`	Shared version for the last three mazes. The same controller structure is used with parameter values suitable for these layouts.

4.2. Main File Locations

The important logic is separated by purpose. This makes the implementation easier to inspect because each file has a specific role. Table 3 gives the main file locations and their function.

Table 3. Main controller files and their purpose.

File name	Purpose in the controller
`manual_mapping.py`	Entry point. It creates the robot, sensors, localization, SLAM, perception, planning, motion, recovery, persistence, state machine, and visualization objects.
`config.py`	Stores map size, robot dimensions, speed limits, colour thresholds, planner weights, recovery thresholds, and floating-wall settings.
`sensors.py`	Enables and reads Webots motors, wheel encoders, IMU, LiDAR, RGB camera, depth camera, and distance sensors.
`localization.py`	Estimates the robot position and heading from wheel motion, IMU yaw, and scan-to-map correction.
`slam.py`	Maintains the occupancy grid, lidar ray tracing, special map layers, and pose-graph mapping support.
`perception.py`	Detects blue/yellow pillars, green floor, lidar pillar candidates, and floating-wall observations.
`floating_fusion.py`	Combines repeated floating-obstacle evidence before stamping it as an obstacle in the map.
`planner.py`	Creates the inflated cost map, selects exploration frontiers, checks paths, and plans routes with A*.
`motion.py`	Converts a route or target into wheel speeds and checks local safety during movement.
`recovery.py`	Detects poor progress and performs reverse, turn, and escape actions when the robot is stuck.
`state_machine.py`	Enforces the mission order from startup and exploration to blue, yellow, and completion.
`visualization.py`	Draws the live map, robot pose, route, trail, pillars, and detected obstacle layers.
`persistence.py`	Records coordinate history and useful run outputs.

The same file names appear inside each controller folder. For example, the Maze 2 planner is located at

`manual_mapping_maze2_solution/manual_mapping/planner.py`, while the shared planner for Mazes 3, 4, and 5 is located at `manual_mapping_maze345_solution/manual_mapping/planner.py`.

4.3. Runtime Pipeline

The control loop runs once per Webots time step. The main data flow is:

Sensors → Pose estimation → Map update → Object detection → State machine → A* planner → Wheel commands

During a normal step, the controller performs these operations:

1. read wheel, IMU, LiDAR, RGB camera, and depth camera data;
2. update the robot pose;
3. update the occupancy map and special map layers;
4. detect blue/yellow pillars, green floor, and floating obstacles;
5. let the state machine decide whether to explore, go to blue, go to yellow, recover, or stop;
6. plan or update a route;
7. send safe wheel speeds to the robot;
8. update the debug map visualization.

5. Mapping and Localization Methodology

5.1. Occupancy Grid Mapping

The mapping method is based on an occupancy grid. This is a suitable method because the mazes are bounded and mostly planar. Occupancy grids are widely used for mobile robot perception because they represent free, occupied, and unknown space in a simple grid form [6].

In the submitted configurations, the map uses a grid size of 1000×1000 cells and a resolution of 0.02 m per cell. The robot is placed near the center of the grid. A cell with free evidence is treated as driveable, a cell with obstacle evidence is treated as blocked, and an unknown cell can become a future exploration target.

5.2. Wheel and IMU Pose Estimation

The robot pose is represented by (x, y, θ). The wheel encoders estimate how far the robot has moved, while the IMU gives the yaw angle. For a small movement Δs, the position update is:

$$x_{t+1} = x_t + \Delta s \cos(\theta), \qquad y_{t+1} = y_t + \Delta s \sin(\theta). \tag{1}$$

This pose is used by mapping, perception, planning, and movement. For example, a LiDAR endpoint can only be placed into the map after the robot pose is known.

5.3. Scan Matching and Pose Graph Support

Wheel odometry can drift over time. The submitted code includes scan matching and pose-graph modules to reduce this drift. The scan matcher converts LiDAR ranges into 2-D points and aligns scans using iterative closest point matching, which is a standard point-set alignment method [7]. The pose-graph files store robot poses and constraints, and the graph optimizer reduces accumulated error when useful constraints are available.

This method is kept lightweight. The robot does not need a full offline map. It only uses corrections that are supported by its own sensor observations.

5.4. LiDAR Ray Tracing

LiDAR is the main sensor for normal wall mapping. For every valid LiDAR beam, the endpoint is marked as occupied and the cells between the robot and the endpoint are marked as free. This creates the visible corridor structure in the map. Repeated observations make walls more stable and free-space areas clearer.

6. Semantic Perception Methodology

6.1. Blue and Yellow Pillars

The RGB camera is used to detect the blue and yellow pillars. The image is converted to HSV colour space, and colour thresholds are applied. The controller does not accept a target only from one colour observation. It checks distance and consistency using depth or LiDAR information and confirms the target over repeated frames. This reduces false detections from reflections, coloured patches, or partial views.

6.2. Green Floor

Green floor is a forbidden area. The controller detects green pixels in the camera image and projects them into the map using distance information. These cells are stored separately from normal walls. In the planner, green cells are blocked like obstacles. OpenCV operations are used for practical colour-mask processing and image cleanup [8].

6.3. Floating Obstacles

Floating obstacles are handled by combining camera, depth, LiDAR, and movement evidence. The depth camera gives height and distance information. The `floating_fusion.py` file combines repeated observations before an obstacle is accepted. This avoids stamping a permanent obstacle from one noisy frame while still allowing the robot to block real obstacles that LiDAR alone may miss.

7. Navigation Methodology

7.1. Cost Map Construction

The planner converts the map into a cost map. Occupied cells, green cells, and confirmed floating obstacles are blocked. Cells near obstacles receive higher cost so that the robot prefers safer positions when the corridor is wide enough. This is a practical compromise: the robot should keep clearance from walls, but it must still pass through valid narrow corridors.

7.2. Frontier-Based Exploration

When no safe route to the current target is known, the robot explores. A frontier is a boundary between known free space and unknown space. Moving toward frontiers helps the robot reveal more of the maze [3]. The planner ranks frontiers using reachability, distance, and usefulness.

7.3. A* Path Planning

The route planner uses A* search on the cost map. A* evaluates a candidate cell with

$$f(n) = g(n) + h(n), \tag{2}$$

where $g(n)$ is the cost from the start to the current cell and $h(n)$ is a heuristic estimate from the current cell to the goal [4]. PythonRobotics was also used as a general robotics reference for understanding grid-based planning, mapping, and path-tracking concepts [5]. In this project, blocked cells are not allowed, and cells near obstacles are more expensive. This makes the path safer than a shortest-path-only route.

7.4. Path Following

The motion controller follows the planned route by turning toward a lookahead point and driving forward when the heading error is acceptable. This is similar to pure-pursuit path tracking, where the robot tracks a point ahead on the path instead of only the closest point [9, 5]. The controller also checks front, side, rear, and corner clearance before sending motor commands.

8. Mission State Machine

The mission order is controlled by `state_machine.py`. The important states are:

$$\texttt{STARTUP} \rightarrow \texttt{PRESCAN} \rightarrow \texttt{EXPLORE} \rightarrow \texttt{NAV_BLUE} \rightarrow \texttt{NAV_YELLOW} \rightarrow \texttt{DONE}$$

In `STARTUP` and `PRESCAN`, the robot initializes the sensors and gathers early map information. In `EXPLORE`, it searches for useful unknown areas and target evidence. In `NAV_BLUE`, the robot tries to reach the blue pillar. Only after blue is reached does the controller change to `NAV_YELLOW`. In `DONE`, the motors are stopped and the final timing is printed.

This state order is important because it directly enforces the assignment rule. The yellow target cannot finish the mission before the blue target has been reached.

9. Safety and Recovery Methodology

9.1. Safety Checks

The controller does not only trust the global path. It also checks local sensor information while moving. The safety checks include front clearance, side clearance, corner clearance, near-obstacle cost, green-floor blocking, and floating-wall evidence. If a planned path becomes unsafe, the planner is asked for a new route.

9.2. Stuck Detection

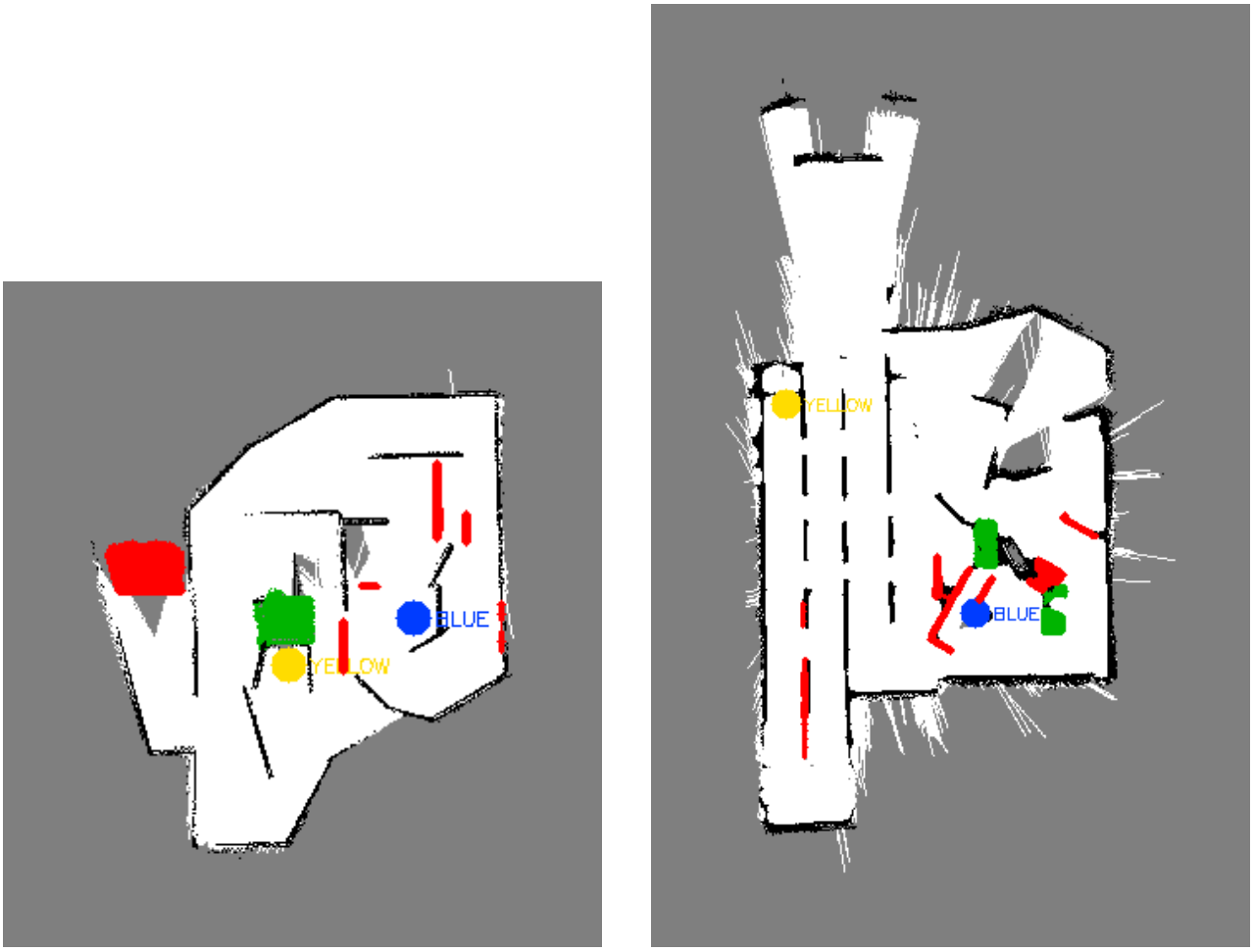
The recovery manager compares expected movement with actual movement. If the robot is commanded to move but the pose changes too little for several steps, it is treated as a stuck situation. This can happen because of a narrow wall contact, a hidden obstacle, or a path that is no longer suitable.

9.3. Recovery Actions

Recovery actions are staged. The robot can stop, reverse, rotate toward open space, nudge forward, or request a different route. The Maze 2 version contains extra recovery and target-handling logic because its recorded run needed more careful movement and route checking.

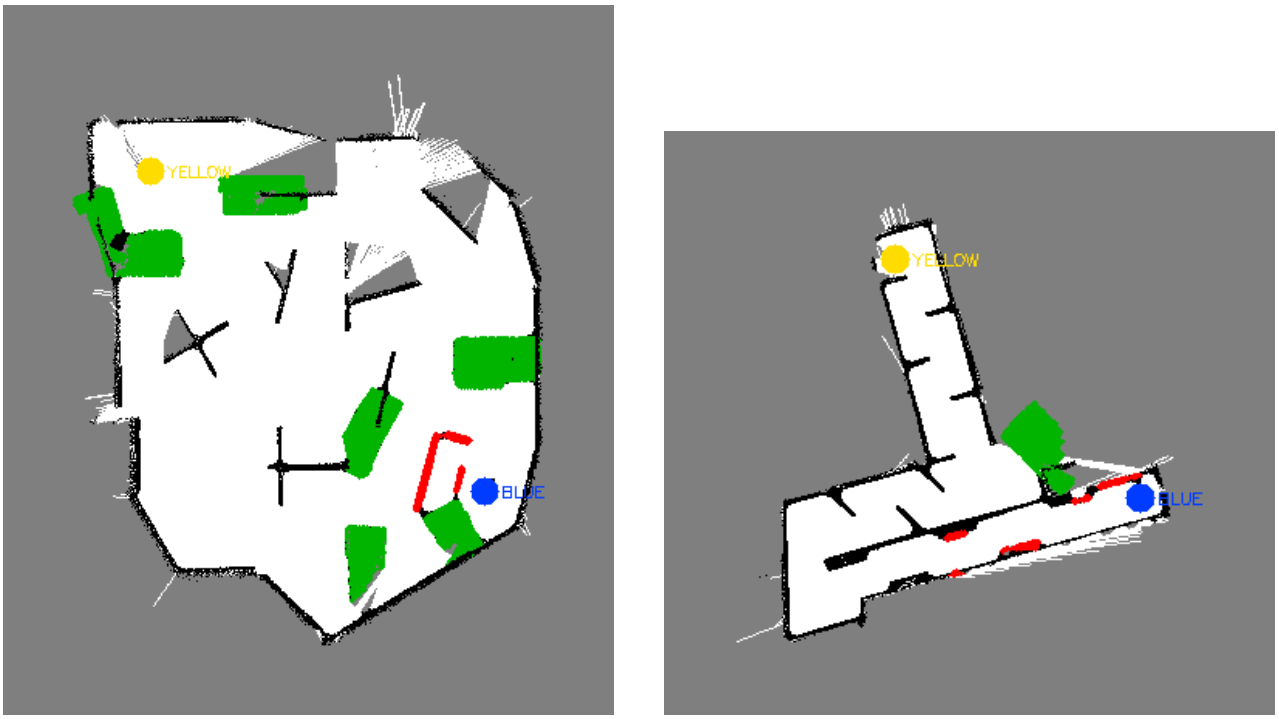
10. Map Outputs

The final detected maps from the five recorded runs are shown in Figures 1–3. In these maps, white areas represent explored free space, black areas show walls or blocked structures, green regions show forbidden floor, and blue/yellow markers show the detected target pillars. Some red markings represent additional blocked or risky cells detected by the controller.



(a) Maze 1 detected map. **(b)** Maze 2 detected map.

Figure 1. Detected maps for Maze 1 and Maze 2.



(a) Maze 3 detected map. **(b)** Maze 4 detected map.

Figure 2. Detected maps for Maze 3 and Maze 4.

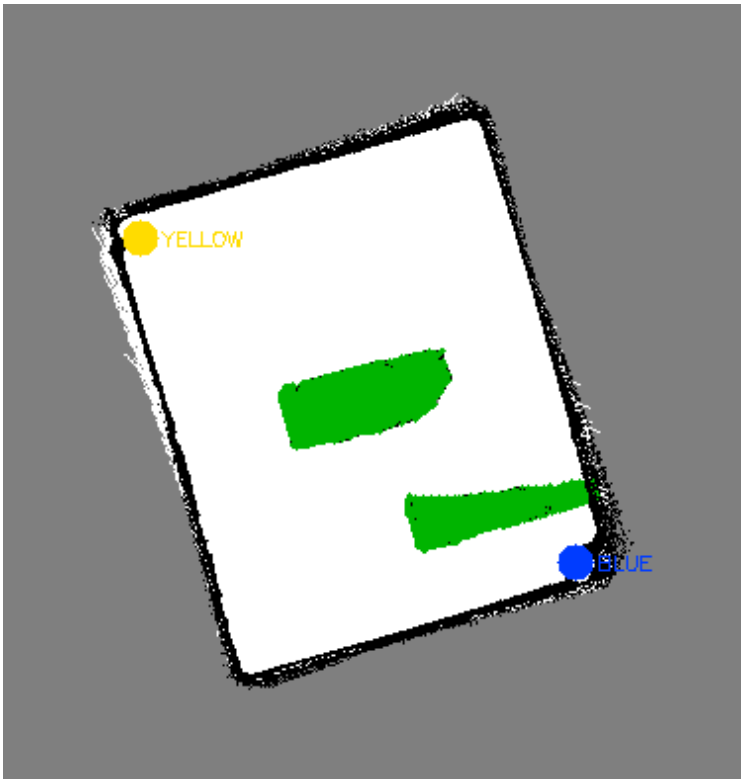


Figure 3. Detected map for Maze 5.

11. Maze-Specific Controller Use

The same navigation principle was used for all five mazes, but the submitted archive contains separate folders for different layouts. The tuning is summarized in Table 4. The descriptions focus only on the actual project behavior.

Table 4. Maze-specific controller use.

Maze	Main folder	Observed reason for this version
Maze 1	`manual_mapping_maze1_solution/manual_mapping/`	The maze was solved using a dedicated version with stable map building, target confirmation, A* routing, and recovery settings for this layout.
Maze 2	`manual_mapping_maze2_solution/manual_mapping/`	This was the longest recorded run. The dedicated version includes additional route checking, map cleanup, recovery, and target handling for a difficult layout.
Maze 3	`manual_mapping_maze345_solution/manual_mapping/`	The shared version handled the larger explored area using the same mapping, perception, planning, and recovery pipeline.
Maze 4	`manual_mapping_maze345_solution/manual_mapping/`	The shared version handled narrow-route behavior and replanning during the run.
Maze 5	`manual_mapping_maze345_solution/manual_mapping/`	The shared version solved this maze fastest because a usable target route became available quickly after mapping and detection.

12. Implementation Summary

The implementation uses the Webots Python controller interface for motors and sensors. NumPy is used for numeric map operations. OpenCV is used for camera image masks and visualization. The algorithm choices are standard mobile-robotics components, and PythonRobotics was used as a background reference for the general ideas of mapping, path planning, and path tracking [5]. The Python files are grouped by responsibility, so the complete controller can be understood as cooperating modules instead of one large script.

The entry point for every submitted controller folder is `manual_mapping.py`. The navigation logic is distributed mainly across `slam.py`, `perception.py`, `planner.py`, `motion.py`, `recovery.py`, and `state_machine.py`. The configuration values are kept in `config.py`, which makes parameter tuning easier to inspect.

13. Testing and Evaluation Methodology

13.1. Mission Success Criteria

A run was counted as successful only if the robot reached the blue pillar first, then reached the yellow pillar, and printed the mission-complete timing. The controller state machine enforces this order internally by changing to yellow navigation only after the blue target is reached.

13.2. Timing Measurement

The recorded timing uses the same format for all mazes. Timer 1 measures simulation start to blue. Timer 2 measures blue to yellow. The total time measures simulation start to yellow. The results are shown in Table 5.

14. Results

Table 5. Recorded timing results.

Maze	Start–Blue	Blue–Yellow	Total	Status
Maze 1	2:51 (171 s)	1:01 (61 s)	3:52 (232 s)	Solved
Maze 2	3:29 (209 s)	1:44 (104 s)	5:13 (313 s)	Solved
Maze 3	2:02 (122 s)	0:36 (36 s)	2:38 (158 s)	Solved
Maze 4	1:23 (83 s)	1:02 (62 s)	2:25 (145 s)	Solved
Maze 5	0:51 (51 s)	0:27 (27 s)	1:18 (78 s)	Solved

Maze 5 was the fastest recorded run with a total time of 1:18. Maze 2 was the longest recorded run with a total time of 5:13. The longer time in Maze 2 is consistent with the need for more route checking, recovery, and careful target handling in that controller folder.

15. Limitations

The controller solved all five recorded runs, but it still has limitations. Colour thresholds depend on lighting and camera angle. Pillar detection can be delayed if a target is only partly visible. Floating-obstacle detection depends on enough useful depth evidence. Very tight passages can still require recovery behavior. The submitted solution also uses separate parameter tuning for different maze layouts instead of automatically learning all parameters.

16. Academic Integrity and Project Constraints

All final controller decisions, integration, testing, and submitted results belong to this project group. The solution did not use the Webots Supervisor API for hidden world information, did not change the provided robot model, and did not modify the provided maze worlds.

17. AI Usage

ChatGPT was used for code debugging and code optimization. It helped to understand error messages, compare possible fixes, and improve the readability of the controller code. The final code changes were tested and integrated by the project group.

Perplexity was used for research and for understanding project-related concepts such as mapping, path planning, robot navigation, and controller design. The final implementation decisions, report structure, and submitted results were checked by the project group.

References

[1] Cyberbotics Ltd. *Webots Reference Manual: Robot*. Accessed for Webots Robot controller interface reference. 2026. URL: https://cyberbotics.com/doc/reference/robot.

[2] Cyberbotics Ltd. *Webots Reference Manual: Supervisor*. Used only to clarify that Supervisor provides privileged world-level functions not used by the submitted controller. 2026. URL: https://cyberbotics.com/doc/reference/supervisor.

[3] Brian Yamauchi. "A Frontier-Based Approach for Autonomous Exploration". In: *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation*. 1997, pp. 146–151. DOI: 10.1109/CIRA.1997.613851.

[4] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: 10.1109/TSSC.1968.300136.

[5] Atsushi Sakai et al. *PythonRobotics: Python Code Collection of Robotics Algorithms*. General reference for robotics algorithms such as mapping, grid-based path planning, and path tracking. 2026. URL: https://github.com/AtsushiSakai/PythonRobotics.

[6] Alberto Elfes. "Using Occupancy Grids for Mobile Robot Perception and Navigation". In: *Computer* 22.6 (1989), pp. 46–57. DOI: 10.1109/2.30720.

[7] Paul J. Besl and Neil D. McKay. "A Method for Registration of 3-D Shapes". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (1992), pp. 239–256. DOI: 10.1109/34.121791.

[8] Gary Bradski. "The OpenCV Library". In: *Dr. Dobb's Journal of Software Tools* (2000).

[9] R. Craig Coulter. *Implementation of the Pure Pursuit Path Tracking Algorithm*. Tech. rep. CMU-RI-TR-92-01. Carnegie Mellon University, Robotics Institute, 1992.